lab05

Fine-Tuning & Customization¶

You'll learn: how to adapt pre-trained LLMs for domain-specific use, explore parameter-efficient methods (LoRA), run a quick custom text-generation task, and evaluate fine-tuned outputs.

Prereqs: Python, Hugging Face transformers, Colab (GPU strongly recommended). **Deliverables**: screenshots of training logs, sample generations, and short reflections.

Lab 1 — Quick Domain Adaptation with Prompt Tuning (20–25 mins)

Goal: See how a simple "prompt prefix" can specialize a general model without retraining.

Setup

In [ ]:

```
from transformers import pipeline

gen = pipeline("text-generation", model="distilgpt2")
```

Steps

1. **Baseline**:

In [ ]:

```
print(
    gen(
        "Summarize: The airplane underwent routine maintenance
today.",
        max_new_tokens=40,
    )
)
```

2. **Apply a domain prefix**:

In [ ]:

```
print(
    gen(
        "As an aviation safety report, summarize: The airplane
underwent routine maintenance today.",
```

```
        max_new_tokens=40,
    )
)
```

3. Compare outputs with/without prefix.
4. Try 2 more prefixes (e.g., "As a legal contract…", "As a customer service email…").

Checkpoint: screenshots of before/after generations.

Reflection: How much did adding context shift style?

In [ ]:

```
print(
    gen(
        "As a legal contract, summarize: The airplane underwent
routine maintenance today.",
        max_new_tokens=40,
    )
)
```

In [ ]:

```
print(
    gen(
        "As a customer service email, summarize: The airplane
underwent routine maintenance today.",
        max_new_tokens=40,
    )
)
```

Lab 2 — Parameter-Efficient Fine-Tuning with LoRA (30–40 mins)¶

Goal: Use Hugging Face peft to adapt a model with LoRA on a small dataset.

Setup

In [ ]:

```
#!pip install -q transformers datasets peft accelerate bitsandbytes
```

Steps

1. Load dataset (SST-2 for sentiment, or your own small CSV).

In [ ]:

```
from datasets import load_dataset

ds = load_dataset("sst2")
```

2. Load model with peft

In [ ]:

```
from transformers import AutoModelForSequenceClassification,
AutoTokenizer
from peft import LoraConfig, get_peft_model

tok = AutoTokenizer.from_pretrained("distilbert-base-uncased")
model = AutoModelForSequenceClassification.from_pretrained(
    "distilbert-base-uncased", num_labels=2
)
config = LoraConfig(
    r=8, lora_alpha=16, target_modules=["q_lin", "v_lin"],
lora_dropout=0.1
)
model = get_peft_model(model, config)
```

3. Fine-tune on a subset (few hundred examples).

4. Evaluate accuracy on validation set.

Checkpoint: sample training log + accuracy printout.

Reflection: Why does LoRA save memory vs full fine-tuning?

Lab 3 — Full Fine-Tuning on Custom Text (40–50 mins)¶

Goal: Run a small-scale fine-tune of GPT-2 on a custom corpus.

Setup

In [ ]:

```
from datasets import Dataset

texts = [
    "We regret to inform you that your flight has been delayed.",
    "Thank you for flying with us. Your boarding gate is A12.",
    "Weather disruptions may cause schedule changes.",
]
dataset = Dataset.from_dict({"text": texts})
```

Steps

1. Tokenize:

In [ ]:

```
tok = AutoTokenizer.from_pretrained("distilgpt2")
```

```
def encode(ex):
    return tok(ex["text"], truncation=True)


ds = dataset.map(encode)
```

2. Use Trainer with AutoModelForCausalLM.
3. Train for 1–2 epochs (tiny dataset = fast).
4. Generate sample output with custom style.

Checkpoint: "before" vs "after" generations.

Reflection: Did outputs pick up your dataset style? What limitations did you notice?

Lab 4 — Evaluation & Comparison (25–30 mins)¶

Goal: Compare base vs customized models with simple metrics.

Steps

1. Generate text with base GPT-2 and fine-tuned GPT-2.
2. Compute BLEU for similarity to reference (e.g., expected summaries).

In [ ]:

```
#!pip install -q sacrebleu

import sacrebleu

refs = [["Your flight has been delayed due to weather."]]
sys = ["We regret to inform you that your flight is delayed."]
print(sacrebleu.corpus_bleu(sys, refs).score)
```

3. Manually compare tone/accuracy.

Checkpoint: printed BLEU + qualitative notes.

Reflection: Which evaluation felt more informative — automated metrics or human inspection?

Grading Rubric (10 pts)

- Prompt tuning outputs (2 pts)
- LoRA setup + training logs (3 pts)
- Custom fine-tune outputs (3 pts)
- Evaluation reflection (2 pts)